

报告编号:C47BFD194DFB4B43AAB736A8A3662707

送检文档:毕业论文终版

作者:刘哲

送检单位:湖南人文科技学院

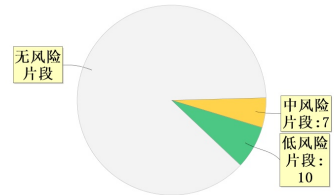
送检时间:2025-05-18 19:20:18

检测结果

疑似AIGC占比: **16.39%** **低风险**

疑似AIGC片段: **17**

疑似AIGC片段 = 高风险片段+中风险片段+低风险片段



片段汇总列表

序号	送检片段	疑似AIGC概率
1	本项目基于 uni-app 框架进行即时通讯应用的设计与实现，旨在打造一款跨平台、高效便捷的即时通讯软件。uni-app 框架具备一次编写、多端运行的特性，大大提高了开发效率。在设计过程中，充分考虑了用户需求和交互体验，采用 WebSocket 协议实现实时消息传输，利用数据库存储聊天记录等数据。实现阶段，通过精心编写代码，完成了用户注册登录、好友管理、群组聊天、一对一聊天、消息发送与接收等核心功能。经测试，该即时通讯项目在多个平台上均可稳定运行，能够满足用户基本的即时通讯需求，具有一定的实用价值和推广前景，为跨平台即时通讯应用开发提供了有效的参考范例。	低
2	即时通讯应用作为人们日常沟通交流的重要工具，在社交、工作、学习等各个领域都发挥着至关重要的作用。人们对于即时通讯的需求也日益多样化和个性化，不仅要求能够实现文字、语音、图片等多种形式的信息交流，还希望具备高效稳定的性能、丰富的社交互动功能以及良好的跨平台使用体验。	低
3	基于 uni - app 框架开发即时通讯项目，能够为跨平台开发领域提供新的实践案例和理论参考。探索在不同平台上实现即时通讯功能的最佳实践，有助于完善跨平台开发的技术体系和方法论。	低
4	实现跨平台的即时通讯应用，用户可以在不同的设备和操作系统上无缝使用，获得一致的用户体验。同时，简洁易用的界面设计和丰富的功能模块，能够满足用户多样化的沟通需求，提升用户的满意度和忠诚度。	中

序号	送检片段	疑似AIGC概率
5	WebSocket、MQTT 等实时通讯协议在国内即时通讯领域得到了广泛应用，确保了消息的实时性和稳定性。同时，国内企业也在不断探索和研发新的实时通讯技术，以满足日益增长的用户需求。	中
6	随着即时通讯行业的快速发展，国家对该行业的监管也日益加强。政府出台了一系列政策法规，加强对即时通讯应用的内容管理、信息安全、隐私保护等方面的监管，确保行业的健康有序发展。	中
7	随着全球范围内对个人隐私和数据安全问题的关注度不断提高，国外用户对即时通讯应用的隐私保护功能要求越来越高。他们希望应用能够提供端到端加密、匿名聊天、隐私设置等功能，确保自己的通讯内容不被第三方获取和监控。	低
8	尽管市场上有众多知名的即时通讯应用，但在一些细分领域仍存在空白。例如，针对特定行业、特定兴趣群体的即时通讯需求尚未得到充分满足。基于 uni - app 开发的项目可以精准定位这些细分市场，提供个性化的服务，从而获得一定的市场份额。	低
9	国内外即时通讯市场均被巨头企业所垄断，如国内的微信、QQ，国外的 WhatsApp、Facebook Messenger 等。这些巨头拥有庞大的用户基础和强大的品牌影响力，新开发的应用很难在短时间内与之竞争。项目需要在功能、用户体验、营销等方面下足功夫，才能吸引用户从现有应用中转移过来。	低
10	即时通讯技术发展迅速，新的协议、算法和功能不断涌现。基于 uni - app 开发的项目需要紧跟技术发展趋势，及时更新和优化应用，以保证其性能和功能的竞争力。同时，还需要应对不同平台的技术差异和兼容性问题。	低
11	随着全球对隐私和安全问题的关注度不断提高，用户对即时通讯应用的隐私保护和信息安全提出了更高的要求。通过各种加密技术，保护了用户隐私内容不被侵犯，这是很重要的。此外，还需要遵守国内外相关的隐私法规和政策，避免因违规而面临法律风险。	中
12	Redis 是一个开源的、基于内存的数据结构存储系统，Redis 具有极高的读写速度，能快速处理大量并发请求，适用于对性能要求极高的场景。同时，它还提供了丰富的功能，如事务、发布订阅、Lua 脚本等，并且具备良好的可扩展性和高可用性，通过集群和复制等机制可以保证数据的可靠性和系统的稳定性，在互联网应用、缓存系统、实时数据处理、分布式锁、消息队列等众多领域都有着广泛的应用。	低
13	Uni - app 作为一款跨平台开发框架，凭借 Vue.js 语法，能让开发者编写一套代码并发布到多平台，如微信小程序、支付宝小程序、H5 以及各类移动 App 等。其丰富的组件库和完善的 API 接口，可充分满足即时通讯项目的开发需求，像界面布局、网络请求处理、本地数据存储等功能都能轻松实现。并且，社区活跃度高，技术文档详尽，开发者在遇到问题时能快速获取解决方案，大大降低了开发难度和技术门槛。	中

序号	送检片段	疑似AIGC概率
14	当下，即时通讯技术已相当成熟，WebSocket 和 MQTT 等协议被广泛应用。 WebSocket 支持在单个 TCP 连接上进行全双工通信，能实现消息的实时、高效传输，非常契合即时通讯场景。同时，市场上众多云服务提供商（如融云、环信）提供了成熟的即时通讯 SDK，这些 SDK 封装了复杂的通讯逻辑，开发者只需简单集成，就能快速实现消息发送、接收、推送等核心功能，节省了大量的开发时间和精力。	中
15	WebSocket适用于多种环境，不仅限于网页浏览器和服务器间的通信，还可以用于其他任何支持TCP协议的应用程序间进行通信。这意味着WSS可以在广泛的平台上实现，包括但不限于移动应用、桌面应用以及各种服务端技术。	中
16	此外，优化WebSocket应用性能的一些策略包括：合理设置心跳机制以保持连接活跃，减少因网络问题导致的连接中断；有效管理服务器端资源，避免因大量并发连接导致的服务过载；以及根据实际应用场景调整消息发送策略，确保重要信息能够及时准确地推送给用户。通过这些措施，可以进一步提升基于WebSocket开发的实时应用的稳定性和用户体验。用户聊天设计页面如图5-4所示：	低
17	随着测试与优化工作的不断推进，该即时通讯项目已具备了一定的实用性与稳定性。然而，技术发展日新月异，即时通讯领域的用户需求也在持续演变。未来，我们将持续关注前沿技术，对项目进行迭代升级，进一步提升系统性能、拓展功能特性，致力于为用户打造更加高效、便捷且安全的即时通讯体验，使其在激烈的市场竞争中保持优势，为即时通讯技术的发展贡献一份力量。	低

全文标红报告



湖南人文科技学院
本科生毕业设计

项目名称：	基于uni-app的即时通讯项目的设计与实现		
学院名称：	信息学院		
学生姓名：	刘哲	学号	214201142
专业年级：	网络工程2021级		

指导教师：	朱素英
教师职称：	副教授
企业导师	

湖南人文科技学院教务处
二〇二四年制
毕业设计原创性及知识产权声明
本人郑重声明：所呈交毕业设计是本人在导师指导下独立进行研究所取得的成果，对本设计的研究做出重要贡献的个人和集体，均已在文中明确标明。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。
本毕业设计成果归湖南人文科技学院所有。
作者签名： 2025 年 月 日

刘哲

导师签名： 2025 年 月 日
朱素英

目 录

摘要·I

Abstract·II

1 前言·1

1.1 研究背景·1

1.2 研究意义·1

1.3 国内现状·2

1.4 国外现状·2

1.5 个人见解·2

1、机遇：·2

2、挑战：·3

2 开发环境与开发技术简介·4

2.1 系统设计的技术选择·4

2.2 系统的运行环境·4

2.3 软件环境·4

2.3.1 Visual Studio Code·4

2.3.2 Node.js·4

2.3.3 MySQL·4

2.3.4 Redis·5

2.4 体系结构·5

3 可行性与需求分析·6

3.1 可行性分析·6

3.1.1 开发框架适用性·6

3.1.2 即时通讯技术可行性·6

3.1.3 经济可行性·6

3.2 需求分析·6

3.2.1 功能需求·6

3.2.2 性能需求·7

3.2.3 环境需求	7
3.2.4 用户界面需求	7
3.2.5 其他需求	7
3.3 业务结构	8
4 概要设计	10
4.1 功能设计	10
4.2 系统结构设计	10
4.3 数据库设计	10
4.3.1 概念设计	10
4.3.2 逻辑结构设计	11
4.3.3 表结构设计	13
5 详细设计	16
5.1 用户管理	16
5.2 联系人管理	18
5.3 即时通讯	19
5.4 群组管理	22
5.5 朋友圈	24
6 系统测试	25
6.1 功能测试	25
6.2 性能测试	26
6.2.1 测试指标	26
6.2.2 测试工具	26
结束语	27
参考文献	28
致谢	29

基于uni-app的即时通讯项目的设计与实现

摘要：

本项目基于 uni-app 框架进行即时通讯应用的设计与实现，旨在打造一款跨平台、高效便捷的即时通讯软件。uni-app 框架具备一次编写、多端运行的特性，大大提高了开发效率。在设计过程中，充分考虑了用户需求和交互体验，采用 WebSocket 协议实现实时消息传输，利用数据库存储聊天记录等数据。实现阶段，通过精心编写代码，完成了用户注册登录、好友管理、群组聊天、一对一聊天、消息发送与接收等核心功能。经测试，该即时通讯项目在多个平台上均可稳定运行，能够满足用户基本的即时通讯需求，具有一定的实用价值和推广前景，为跨平台即时通讯应用开发提供了有效的参考范例。

关键词：uni-app，WebSocket，数据库，聊天

Design and Implementation of Instant Messaging Project Based on uni-app

Abstract:

This project is based on the uni-app framework for the design and implementation of instant messaging applications, aiming to create a cross-platform, efficient and convenient instant messaging software. The uni-app framework has the characteristics of one-time writing and multi-terminal operation, which greatly improves the development efficiency. In the design process, the user needs and interactive experience are fully considered, the WebSocket protocol is used to realize real-time message transmission, and the database is used to store chat records and other data. In the implementation phase, through carefully written code, the core functions such as user registration and login, friend management, group chat, one-to-one chat, message sending and receiving are completed. After testing, the instant messaging project can run stably on multiple platforms, can meet the user's basic instant messaging needs, has certain practical value and promotion prospects, and provides an effective reference example for the development of cross-platform instant

messaging applications.

Keywords: uni-app,WebSocket, database, chat

前言

研究背景

即时通讯应用作为人们日常沟通交流的重要工具，在社交、工作、学习等各个领域都发挥着至关重要的作用。人们对于即时通讯的需求也日益多样化和个性化，不仅要求能够实现文字、语音、图片等多种形式的信息交流，还希望具备高效稳定的性能、丰富的社交互动功能以及良好的跨平台使用体验。

随着市场的不断扩大，以及互联网的普及，很多不同的设备的产生，我们需要这种可以兼容不同端的代码，做到一套代码多端复用。

市场上已经有许多的即时通讯的产品，但是大部分都不是开源的，不适合大家去学习即时通讯的应用，而且大部分即时通讯应用商业化严重，所以需要有一个简介的聊天应用，并且可以跨平台。

研究意义

基于 uni-app 框架开发即时通讯项目，能够为跨平台开发领域提供新的实践案例和理论参考。探索在不同平台上实现即时通讯功能的最佳实践，有助于完善跨平台开发的技术体系和方法论。

深入研究即时通讯的核心技术，如实时消息传输、消息存储与管理、用户身份认证等，为该领域的学术研究提供新的思路 and 方向。通过项目实践，验证和改进现有即时通讯技术的理论模型，推动即时通讯技术的发展。

实现跨平台的即时通讯应用，用户可以在不同的设备和操作系统上无缝使用，获得一致的用户体验。同时，简洁易用的界面设计和丰富的功能模块，能够满足用户多样化的沟通需求，提升用户的满意度和忠诚度。

国内现状

国内即时通讯产品已经趋于成熟，微信、QQ等聊天应用完善。不仅满足了人们日常的通讯需求，还集成了支付、购物、生活服务等多种功能，成为了一个综合性的生活平台。

国内开发者对跨平台开发技术的应用日益成熟，除了 uni-app，还有 Taro、React Native 等框架也被广泛使用。这些框架使得开发者能够更高效地开发出多平台兼容的即时通讯应用，降低了开发成本和时间。

WebSocket、MQTT 等实时通讯协议在国内即时通讯领域得到了广泛应用，确保了消息的实时性和稳定性。同时，国内企业也在不断探索和研发新的实时通讯技术，以满足日益增长的用户需求。

随着即时通讯行业的快速发展，国家对该行业的监管也日益加强。政府出台了一系列政策法规，加强对即时通讯应用的内容管理、信息安全、隐私保护等方面的监管，确保行业的健康有序发展。

国外现状

WhatsApp、Facebook Messenger、Telegram 和 LINE 等在国际市场上拥有庞大用户群体。WhatsApp 凭借 Facebook 的推广和自身简洁易用的特点，在欧美、南美、非洲等地区广泛使用，拥有数十亿用户。Facebook Messenger 与社交平台 Facebook 深度整合，方便用户在社交网络基础上进行即时通讯。Telegram 以注重隐私保护和强大的群组功能吸引了大量用户，尤其在一些对隐私较为敏感的地区和特定社群中很受欢迎。LINE 则在日本、韩国等亚洲国家占据重要市场份额，除通讯外，还提供丰富的内容服务，如游戏、表情商店等。

随着全球范围内对个人隐私和数据安全问题的关注度不断提高，国外用户对即时通讯应用的隐私保护功能要求越来越高。他们希望应用能够提供端到端加密、匿名聊天、隐私设置等功能，确保自己的通讯内容不被第三方获取和监控。

1.5个人见解

1、机遇：

当前市场对于跨平台应用的需求十分强烈，无论是国内还是国外，用户都希望能在不同设备和系统上无缝使用即时通讯应用。uni-app 一次开发多端部署的特性，能有效满足这一需求，降低开发成本和周期，使产品能快速推向市场，在竞争中占据先机。

尽管市场上有众多知名的即时通讯应用，但在一些细分领域仍存在空白。例如，针对特定行业、特定兴趣群体的即时通讯需求尚未得到充分满足。基于 uni-app 开发的项目可以精准定位这些细分市场，提供个性化的服务，从而获得一定的市场份额。

2、挑战：

国内外即时通讯市场均被巨头企业所垄断，如国内的微信、QQ，国外的 WhatsApp、Facebook Messenger 等

。这些巨头拥有庞大的用户基础和强大的品牌影响力，新开发的应用很难在短时间内与之竞争。项目需要在功能、用户体验、营销等方面下足功夫，才能吸引用户从现有应用中转移过来。

即时通讯技术发展迅速，新的协议、算法和功能不断涌现。基于 uni-app 开发的项目需要紧跟技术发展趋势，及时更新和优化应用，以保证其性能和功能的竞争力。同时，还需要应对不同平台的技术差异和兼容性问题。

随着全球对隐私和安全问题的关注度不断提高，用户对即时通讯应用的隐私保护和信息安全提出了更高的要求。通过各种加密技术，保护了用户隐私内容不被侵犯，这是很重要的。此外，还需要遵守国内外相关的隐私法规和政策，避免因违规而面临法律风险。

开发环境与开发技术简介

系统设计的技术选择

开发语言：html, css, JavaScript

数据库：mysql, redis

服务器：本地服务器

系统的运行环境

系统要求：Windows10、11

服务器要求：Node18

数据库服务器要求：MySQL ,Redis

软件环境

2.3.1 Visual Studio Code

Visual Studio Code 是微软开发的免费开源跨平台代码编辑器，具有跨平台性、轻量级的特点，拥有丰富的扩展生态，能实现智能代码补全，具备强大的调试功能，集成终端和 Git，适用于软件开发、Web 开发、数据科学、文档编辑等多种场景，可通过安装扩展满足不同用户的多样化需求，在开发者中广受欢迎。

2.3.2 Node.js

Node.js 是一个基于 Chrome V8 引擎的 JavaScript 运行时环境，具有异步非阻塞 I/O、单线程、跨平台以及丰富的模块系统等特点，能让 JavaScript 在服务器端运行，可应用于 Web 应用程序后端、实时应用、命令行工具以及微服务架构等场景，由 Ryan Dahl 于 2009 年创建，经过多年发展已成为现代 Web 开发中极为重要的一部分。

2.3.3 MySQL

Oracle 的 MySQL 是一种数据库管理系统，提供了在不同平台（如 Linux、Microsoft Windows、macOS、Solaris 等）上安装和配置软件的各种方法。MySQL 的安装过程可以通过下载通用软件包或从源代码编译来完成。还可以选择使用 Docker Hub 的社区版或 My Oracle Container Registry 进行安装。

2.3.4 Redis

Redis 是一个开源的、基于内存的数据结构存储系统，Redis 具有极高的读写速度，能快速处理大量并发请求，适用于对性能要求极高的场景。同时，它还提供了丰富的功能，如事务、发布订阅、Lua 脚本等，并且具备良好的可扩展性和高可用性，通过集群和复制等机制可以保证数据的可靠性和系统的稳定性，在互联网应用、缓存系统、实时数据处理、分布式锁、消息队列等众多领域都有着广泛的应用。

体系结构

本系统采用 B/S 架构，前后端完全分离，易于维护和扩展。

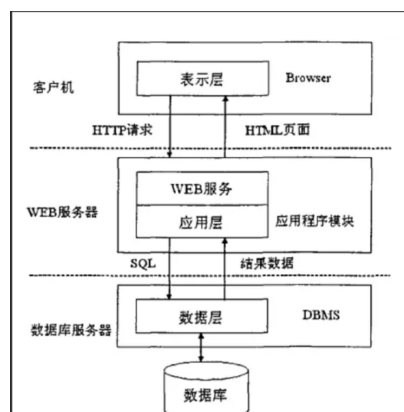


图2-1 B/S架构交互图

3可行性与需求分析

3.1 可行性分析

开发框架适用性

Uni-app 作为一款跨平台开发框架，凭借 Vue.js 语法，能让开发者编写一套代码并发布到多平台，如微信小程序、支付宝小程序、H5 以及各类移动 App 等。其丰富的组件库和完善的 API 接口，可充分满足即时通讯项目的开发需求，像界面布局、网络请求处理、本地数据存储等功能都能轻松实现。并且，社区活跃度高，技术文档详尽，开发者在遇到问题时能快速获取解决方案，大大降低了开发难度和技术门槛。

即时通讯技术可行性

当下，即时通讯技术已相当成熟，WebSocket 和 MQTT 等协议被广泛应用。WebSocket 支持在单个 TCP 连接上进行全双工通信，能实现消息的实时、高效传输，非常契合即时通讯场景。同时，市场上众多云服务提供商（如融云、环信）提供了成熟的即时通讯 SDK，这些 SDK 封装了复杂的通讯逻辑，开发者只需简单集成，就能快速实现消息发送、接收、推送等核心功能，节省了大量的开发时间和精力。

经济可行性

使用 Uni-app 进行开发可以大大降低开发成本，因为只需要编写一套代码就可以发布到多个平台，减少了开发人员的工作量和开发周期。并且现在有很多免费的云服务器供大家测试使用，大大减少了部署的成本。

3.2 需求分析

功能需求

用户管理模块（登录与注册、个人信息修改等）

联系人管理（搜索好友、添加好友、删除好友等）

即时通讯（单聊、群聊等）

群组管理（创建群聊、群聊信息设置和查看、退出群聊等）

朋友圈（发布动态、点赞、评论等）

性能需求

即时性要求：消息发送和接收要具有高度的即时性，一般情况下，消息的发送和接收延迟应控制在 1 - 3 秒内，确保用户能够及时收到和回复消息，实现流畅的实时交流。

应用界面的切换、加载以及操作响应要迅速，如点击聊天列表中的某个对话，应在 1 秒内显示聊天界面并加载出最近的聊天记录；进行添加好友、创建群组等操作时，响应时间也应控制在 2 - 3 秒内，避免用户长时间等待。

随着用户数量的增加，应用需要具备处理高并发的能力。在峰值时段，要能够支持至少 1000 - 5000 个用户同时在线进行消息发送、接收以及音视频通话等操作，保证系统不出现卡顿、崩溃等问题。

环境需求

软件环境需求。需要确保系统能够高效地运行和支持业务需求。

硬件环境需求。需要确保系统能够满足业务需求并具备足够的容量和性能。

用户界面需求

系统须满足的人性化的用户交互和界面设计，以提供良好的用户体验和易用性。

其他需求

可靠性需求。系统需要具备故障恢复、备份和数据恢复等机制，以确保系统的可靠性和连续性。

安全性需求。系统需要具备某些重要信息的加密功能，包括用户密码，重要文件数据等信息。

可移植性需求。代码风格需要统一，包括变量命名、函数命名、文件名称命名，以及代码的可读性等。

可维护性需求。系统需要遵循合适的编程规范和设计模式，以确保系统的可维护性和可扩展性。

3.3 业务结构

系统业务流程图，以登陆模块为例，用户在客户端的登录页输入账号和密码进行登录。服务端会对用户的输入进行校验，如果账号和密码都正确，则会生成一个token，并且返回给客户端，客户端收到token后会存储在本地，以便下次快速登录，如果登录失败则提示账号或密码错误，如图3-1：



图3-1 登陆系统流程图

页面梳理：

用户管理模块：登录与注册页、用户搜索页、信息修改页。

联系人管理模块：联系人列表、联系人信息设置、添加联系人。

即时通讯模块：聊天页。

群组管理模块：添加群聊页、群聊设置页、群聊列表页。

朋友圈模块：朋友圈列表页、朋友圈发布页。

4概要设计

4.1功能设计

用户管理模块（登录与注册、个人信息修改等）

联系人管理（搜索好友、添加好友、删除好友等）

即时通讯（单聊、群聊等）

群组管理（创建群聊、群聊信息设置和查看、退出群聊等）

朋友圈（发布动态、点赞、评论等）

4.2系统结构设计

采用分布式架构，将系统拆分为多个独立的服务模块，通过网络进行通信和协作。整体架构主要包括客户端、应用服务器、消息服务器、数据库服务器等部分。客户端使用 Uni-app 开发，可发布到多个平台；应用服务器负责处理业务逻辑，如用户注册、登录、好友管理等；消息服务器专门处理消息的实时传输；数据库服务器用于存储用户信息、聊天记录等数据。

4.3 数据库设计

4.3.1 概念设计

本项目是一个即时通讯的项目，提供一个用户社交的平台，所以需要有用户管理功能，而用户与用户之间是互相关联和相对隔离的，比如某个用户如果想要进行交友，必须先加对方为好友，才可以进行聊天和其他互动功能，所以需要有一个管理用户联系人的表，然后就是聊天消息的发送，需要有一个存储的地方，可以进行永久性和暂时性存储，考虑到用户的隐私性，这里只会进行暂时性存储到redis服务中，并且保存一定的周期，并且聊天分为单聊和群聊，所以这两种消息应该存储到redis的不同key中，为了提高用户的体验和个性化，应该提供用户的个性化设置，包括但不限于名称修改，头像修改，个性签名等等，用户之间需要有一定的互动性，可以增加一定的关联，比如朋友圈的发布、评论、点赞等功能。

4.3.2 逻辑结构设计

综上分析得到的系统的实体与它们之间的关系如下：

1.实体：

用户（users）：包含用户的基本信息（用户ID、用户名称、用户头像等）。

联系人（contacts）：包含用户的联系人的基本信息（用户ID、联系人ID等）。

群聊（groups）：包含所有创建的群聊的基本信息（群聊ID、创建人ID、群聊名称、群聊头像等）。

联系人申请（contactsapplication）：包含请求成为联系人的基本信息（申请人ID、被申请人ID等）。

群聊成员（groupmembers）：包含群聊成员的基本信息（用户ID、用户名称、用户头像等）。

朋友圈（moments）：包含朋友圈的基本信息（发布者ID、发布内容、评论、点赞等）。

2.关系：

用户与联系人的关系：多对多的关系，通过用户与联系人表关联实现。

用户和群聊的关系：多对多的关系，通过用户与群聊表关联实现。

用户和朋友圈的关系：一对多的关系，通过关联用户表中的ID实现。

3.实体属性图：

用户实体属性图如图4-1所示：

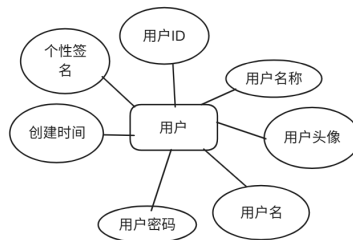


图4-1 用户实体属性图

联系人实体属性图如图4-2所示：

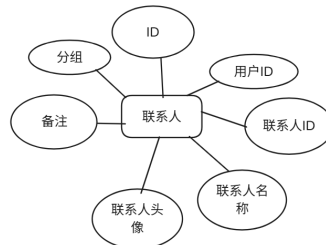


图4-2 联系人实体属性图

联系人申请实体属性图如图4-3所示：

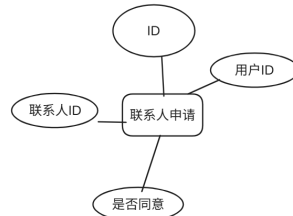


图4-3 联系人申请实体属性图

群组表实体属性图如图4-4所示：

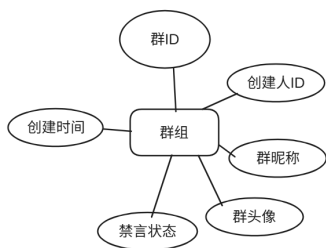


图4-4 群组实体属性图

朋友圈实体属性图如图4-5所示：

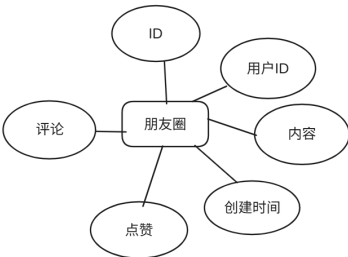


图4-5朋友圈实体属性图

4.数据库ER图如图4-6所示：

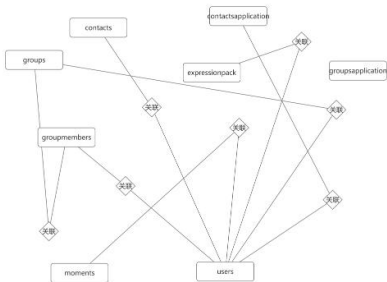


图4-6数据库ER图

4.3.3 表结构设计

如由上面的逻辑结构设计得出的所有表名及字段，根据实际需求得出每一个表的物理结构设计如表4-1-4-7所示。

表4-1 用户表 (users)

字段名	字段类型
id	bigint
name	varchar
avatar	varchar
account	varchar
password	varchar
createTime	bigint
signature	varchar
tags	json

表4-2 联系人表 (contacts)

字段名	字段类型
id	bigint
userId	bigint
contactUserId	bigint
name	varchar

avatar	varchar
grouping	enum

表4-3 联系人申请表 (contactsapplication)

字段名	字段类型
id	bigint
userId	bigint
contactUserId	bigint
agree	tinyint

表4-4 群组表 (groups)

字段名	字段类型
id	bigint
name	varchar
avatar	varchar
ownerId	bigint
silence	tinyint
memberIds	json
createTime	bigint

表4-5 群组成员表 (groupmembers)

字段名	字段类型	长度	主/外键	字段值约束	对应中文名
id	bigint		F	not null	唯一id
name	varchar	30		not null	用户名
avatar	varchar	100		not null	用户头像
groupId	varchar	50		not null	群id
userId	bigint			not null	用户id

表4-6 群组成员续表 (groupmembers)

字段名	字段类型
silence	tinyint
role	enum
joinedTime	bigint

表4-7 朋友圈表 (moments)

字段名	字段类型
id	bigint
userId	bigint
content	varchar
imgList	json
thumbs	json
comments	json
createTime	bigint

5详细设计

用户管理

用户管理包括登录与注册、个人资料查看和修改等。登录模块使用的是JWT技术，是一种token识别机制，在服务端对用户的某些数据进行加密然后生成token，最后返回给前端，前端请求带上token，服务端在对这个token进行解密，如果成功则放行这次请求，否则将阻止这次非法请求，它同样被Base64Url编码形成JWT的第二部分。

签名：要创建签名部分，你需要获取编码后的头部、编码后的载荷、一个密钥 (secret)、头部中指定的算法

。token生成和解析使用的库是jsonwebtoken，具体判断逻辑就是校验本地存放的token，如果token正确且没有过期，则用户处于登录状态，否则为未登录状态，则重定向到登录页。

```
uni.redirectTo({
  url: '/pages/login/index'
});
```

用户登录与注册页面设计如图5-1所示：



图5-1 登录与注册页

个人资料的修改包括头像、名称、个性签名、个性标签等修改，这里有一个要注意的点，就是修改个人资料后需要同步给所有的表，因为很多信息都会与用户进行关联，所以需要同步修改后的信息，这里使用mysql中的触发器完成信息的同步。修改个人信息的部分代码如下：

```
const [rows] = await promisePool.query(
  `UPDATE ${userTable} SET name = ?, avatar = ?, signature = ?, tags = ? WHERE id = ?`,
  [name, avatar, signature, JSON.stringify(sqlTags), userId]
);
```

个人信息修改设计页如图5-2所示:



图5-2 个人信息修改设计页

联系人管理

联系人管理，旨在为用户提供一个便捷的社交网络管理体验。这里的核心逻辑就是根据搜索词搜索目前数据库中注册的用户，然后发送申请后，在联系人申请表中新插入一条记录即可，联系人管理相关页面设计如图5-3所示：



< q | 搜索用户账号/名字/群号 清空



测试

666

keep

发消息

删除好友

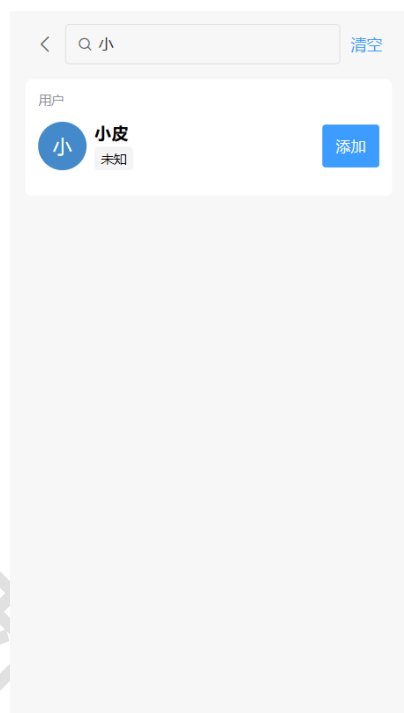


图5-3 联系人管理相关页面设计

搜索好友：根据用户输入的ID或者名称，对数据库进行检索，逻辑如下：

```
const [rows] = await promisePool.query(
  `SELECT id, name, avatar, account, createTime
  FROM ${userTable}
  WHERE account = ? OR name LIKE CONCAT('%', ?, '%')`,
  [searchValue, searchValue]
);
```

发送好友申请：当搜索到用户后，发送好友申请，向服务端发送post请求（携带自己的ID和添加好友的ID），然后服务端处理后向数据库插入一条记录，逻辑如下：

```
const [result] = await promisePool.query(
  `INSERT INTO ${contactsApplicationTable} (contactUserId, userId, name, avatar)
  VALUES (?, ?, ?, ?)
  ON DUPLICATE KEY UPDATE name = VALUES(name), avatar = VALUES(avatar)`,
  [contactUserId, userId, name, avatar]
);
```

被申请添加的用户可以在联系人页面的新朋友列表中查看到申请信息。当用户同意申请后，会向agree-application接口发送一个post请求。服务端接收到请求后，会处理传入的参数，并在contacts联系人表中新增一条记录。逻辑如下：

```
// 使用 ON DUPLICATE KEY UPDATE 处理可能的重复插入
const [result] = await promisePool.query(
  `INSERT INTO ${contactsTable} (contactUserId, userId, name, avatar)
  VALUES (?, ?, ?, ?)
  ON DUPLICATE KEY UPDATE name = VALUES(name), avatar = VALUES(avatar)`,
  [contactUserId, userId, name, avatar]
);
```

即时通讯

即时通讯使用WebSocket协议去完成，他具备长连接的特性，也就是一旦建立连接后，就会在前端和服务端建立一条全双工的通讯通道，前端可以发送数据给服务端，服务端可以发送数据给前端[2]。

这种特效就可以用在这种需要实时通信的场景，建立连接后即可进行实时的通信收发，游戏，聊天应用大部分都是这种技术，比传统的http协议的通信效率高得多，主要是因为它只需要建立一次连接即可。

WebSocket适用于多种环境，不仅限于网页浏览器和服务器间的通信，还可以用于其他任何支持TCP协议的应用程序间进行通信。这意味着WSS可以在广泛的平台上实现，包括但不限于移动应用、桌面应用以及各种服务端技术。

此外，优化WebSocket应用性能的一些策略包括：合理设置心跳机制以保持连接活跃，减少因网络问题导致的连接中断；有效管理服务器端资源，避免因大量并发连接导致的服务过载；以及根据实际应用场景调整消息发送策略，确保重要信息能够及时准确地推送给用户。通过这些措施，可以进一步提升基于WebSocket开发的实时应用的稳定性和用户体验。用户聊天设计页面如图5-4所示：



图5-4 用户聊天设计页

消息类型可以分为文本消息、图片消息、表情包等。为了保护用户的隐私安全，用户之间的聊天消息只会被保存到redis中进行暂时性存储，不会被存储到mysql中。消息状态分为已发送，已送达，已读。为了保证消息在发送过程中的顺序不会错乱，这里会记录每次发送消息的时间戳，然后每次加载消息的时候都会对消息按时间戳进行一次排序，以此保证消息的时序性，并且消息的发送时间也会展示在页面中。

消息发送的逻辑：使用最常用的长连接协议wss进行客户端与服务端的连接，使用websocket库加快开发效率，应用在开始加载页面之前，会向服务端发送wss连接，连接成功后，才能进行后续操作，用户每次发送消息都会带上发送者ID和接收者ID，服务端根据ID去判断这个消息的去向，然后客户端接收这个消息，展示在页面中。那么服务端是如何存储这个ID的呢，其实每次建立wss连接，都会有一个唯一的连接ID，把这个连接ID作为value，而用户ID作为key，就可以完全区分聊天消息的去向了[6]，比如：

```
users[userId] = socket.id;
```

将所有在线的用户的ID存放在users映射表中，每次发送消息就查这个用户映射表即可。

区分消息类型的逻辑：这里不管什么消息类型，其实都只是需要一个key值去区分而已，比如我这里会在每个消息的对象中，使用一个type字段进行区分，如果type是text那就是纯文本，如果是image，那就是图片，区分不同的消息服务端可以做出不同的逻辑处理，而客户端则会根据不同的消息类型进行不同的展示，以下是完整的消息类型的对象：

```
{
  "id",
  "senderId",
  "receiverId",
  "content",
  "type"
}
```

```

    userInfo,
    createTime,
    messageType,
    isMe: false,
    groupId // 明确标识群组消息
  }

```

离线消息存储与发送的逻辑：在发送消息的时候，会判断目前用户是否在线，如果不在线，会先将消息存储到redis中，下面是redis存储消息的逻辑：

```

async function storeMessageInRedis(message) {
  if (!message.createTime) {
    console.error('Cannot store message without createTime', message);
    return;
  }
  // 根据是否存在groupId选择存储键
  let key;
  if (message.groupId) {
    key = `group_messages:${message.groupId}`; // 群聊消息键
  } else {
    key = `messages:${message.senderId}:${message.receiverId}`; // 单聊消息键
  }
  const value = JSON.stringify(message);
  // 存储到有序集合
  await redisClient.zAdd(key, { score: message.createTime, value });
  // 设置过期时间（7天）
  await redisClient.expire(key, 60 * 60 * 24 * 7);
}

```

每当有新的用户上线后，都会去redis中查找这个用户有没有离线消息需要发送，如果有会将redis中的离线消息发送给指定用户。

表情包发送的逻辑：支持gif动图，这里需要使用图片上传的功能，用户将表情包的图片上传到服务端，服务端会自动解析图片的后缀，并且读取内容后，存储到静态目录upload中，此时会生成一个表情包的链接地址返回给客户端，然后展示在页面中，用户发送表情包的时候，会被当成图片消息进行发送，但是其实只是发送的表情包的链接地址

5.4 群组管理

新建群聊：新建群聊必须要至少3位用户，才可以新建，创建者默认为群主，可以进行群昵称、群公告、头像等设置，创建群聊设计页如图5-5所示：



图5-5 创建群聊设计页

创建后群聊成员即可进行聊天服务，创建群聊实际上会操作两个数据库的表，一个就是在群组表中新增一条群聊的记录，然后还会向群聊成员表中新增所有群聊成员的记录，所以必须保证这两种操作都要成功，才返回ok。

群聊的相关设置：群聊目前相关设置包括：头像修改，昵称修改，群公告修改，踢人，退出群聊等，后续会进行完善，群聊设置页设计如图5-6所示：



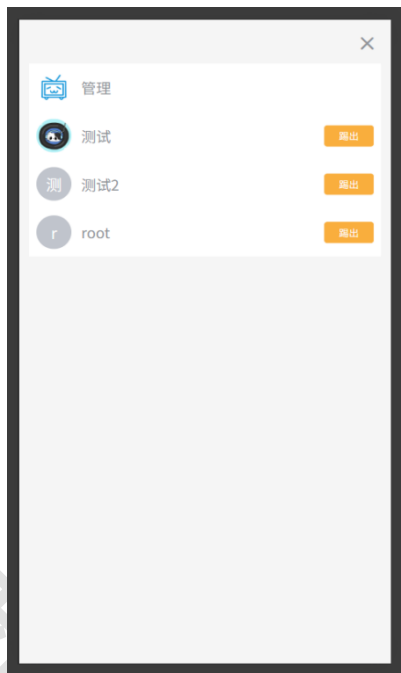


图5-6 群聊设置页

踢出功能逻辑：首先踢出功能一定要做权限管理，目前只有群主具备踢人功能，普通成员踢人服务端会直接拦截，拦截逻辑就是判断群的创建者与当前发起踢人请求的userId是否一致，如果一致那么就判定为群主，则执行后续踢人逻辑，否则会直接return进行拦截。

5.5 朋友圈

朋友圈功能需要展示所有好友发布的朋友圈信息，用户可以发布自己的朋友圈动态，并且支持图片上传。此外，还应具备评论和点赞等核心互动功能，以增强用户之间的互动和社交体验。朋友圈设计页如图5-7所示：





图5-7 朋友圈设计页

朋友圈展示逻辑：由于朋友圈只会展示给好友看，所以每条朋友圈都会有userId, 然后再查找联系人表，判断这条朋友圈的userId是否在联系人ID的数组里面，如果在才会展示。

发布动态逻辑：发布动态由于需要支持图片发布，所以如果用户发布图片的时候，首先会有一个临时地址，展示在页面中，发布后才会进行上传，并且将动态内容一起上传，最后再操作数据库新增一条信息。

评论和点赞逻辑：对与点赞和评论，thumbs和 comments字段进行存储，他们是一个json数组的格式，每次点赞都会更新这两个字段

6系统测试

功能测试

系统功能测试是软件测试的一个方面，专注于验证软件系统的各个功能是否按照预期工作。它基于软件需求规格说明书定义的功能点，检查系统能否正确地完成每个功能操作。系统功能测试主要关注的是软件对外提供的业务逻辑、用户界面交互以及数据处理等方面是否满足用户的需求和期望。

系统功能测试通常分为黑盒测试和白盒测试。

黑盒测试是把软件看作一个不透明的黑盒子，不考虑内部结构和实现原理，仅依据软件的需求规格说明书，检查软件的功能、性能是否符合要求。

白盒测试是基于软件内部的逻辑结构和代码实现进行测试，根据具体代码去写单元测试进行代码bug的修改。

本系统在功能成型后便进行了相应的黑盒测试。

对此我们准备了以下测试用例，如表6-1所示。

表6-1 黑盒测试用例

测试功能名	输入	预期输出	实际输出	结论
用户登录	账号输入“admin”，密码输入“123”	登录成功	登录成功	测试通过
用户注册	名称输入“管理”账号输入“admin”，密码输入“123”	注册成功	注册成功	测试通过
个人信息修改	名称输入“ttt”个性签名输入“ddd”增加个性标签“ccc”	修改成功	修改成功	测试通过
搜索好友	搜索框输入“admin”	成功搜索到admin用户	成功搜索到admin用户	测试通过
添加好友	同意申请	添加成功	添加成功	测试通过
删除好友	点击删除好友	删除成功	删除成功	测试通过

文本消息发送	选择某个联系人发送文本消息	发送成功，好友收到该消息	发送成功，好友收到该消息	测试通过
图片消息发送	选择某个联系人发送图片消息	发送成功，好友收到该消息	发送成功，好友收到该消息	测试通过
动态发布	点击发布动态	朋友圈看到该条动态	朋友圈看到该条动态	测试通过
评论动态	输入“123”	评论成功	评论成功	测试通过
点赞动态	点击点赞图标	点赞成功	点赞成功	测试通过

性能测试

测试指标

响应时间：指从用户发起请求到系统给出响应的时间间隔。在即时通讯项目中，包括发送消息、添加好友、登录等操作的响应时间。一般来说，对于实时性要求较高的操作，如消息发送，响应时间应在 1 秒以内，以提供良好的用户体验。

吞吐量：表示系统在单位时间内处理的请求数量或数据量。例如，在群聊场景中，衡量每秒能够成功发送和接收的消息数量。吞吐量与系统的硬件资源、网络带宽以及软件的性能优化程度等因素密切相关。

并发用户数：指同时访问系统的用户数量。即时通讯应用通常会有大量用户同时在线，需要测试系统在不同并发用户数下的性能表现，以确定系统的承载能力。例如，通过性能测试确定系统能够稳定支持的最大并发用户数，确保在高峰时段系统不会出现崩溃或响应缓慢的情况。

资源利用率：主要包括 CPU 使用率、内存使用率、网络带宽利用率等。

测试工具

JMeter：一款功能强大的开源性能测试工具，可用于模拟大量用户并发访问系统，发送各种类型的请求，并收集性能数据。

LoadRunner：是一款专业的性能测试工具，能够模拟大量用户并发执行各种业务操作，对系统的性能进行全面的评估和分析。

结束语

在本次基于 Uni-app 的即时通讯项目开发与研究过程中，从系统结构设计搭建起稳固框架，到功能测试、黑盒白盒测试以及性能测试多维度把控质量，各个环节紧密相扣，共同推动项目逐步完善。

功能测试凭借精心设计的用例，对用户注册登录、好友管理、消息收发等核心功能逐一核验，确保了用户操作的流畅性与功能实现的准确性。

随着测试与优化工作的不断推进，该即时通讯项目已具备了一定的实用性与稳定性。然而，技术发展日新月异，即时通讯领域的用户需求也在持续演变。未来，我们将持续关注前沿技术，对项目进行迭代升级，进一步提升系统性能、拓展功能特性，致力于为用户打造更加高效、便捷且安全的即时通讯体验，使其在激烈的市场竞争中保持优势，为即时通讯技术的发展贡献一份力量。

参考文献

陆志浩.企业级即时通讯APP及系统设计与实现[J].公安部第一研究所,7~10,2023.

李根.面向电商平台的即时通讯系统设计与实现[D].西安电子科技大学,11~20, 2023.

方杰.基于Linphone的云通信即时通讯软件设计与实现[D]. 北京邮电大学,4~15, 2022

黎雨婷.基于Tor的即时通讯前端子系统的设计与实现[J]. 北京邮电大学6~12 2022.

律方成.基于Tor的并行传输即时通讯系统的设计与实现[J]. 北京邮电大学,13~20,2022.

薛飞跃.万人群即时通讯系统关键技术的设计与实现[J]. 北京交通大学,20~24, 2022.

柯有兴.分布式即时通讯系统的设计与实现[D].华中科技大学 ,1~15, 2020.

万浩然.一个多端消息系统的设计与实现[D]. 华中科技大学,1~10, 2022.

赵志强,于瑞媛, 视频会议系统与即时通讯的综合应用研究[J]. 电子技术与软件工程,8~12, 2021.

谭乾.企业即时通讯平台建设的项目管理研究[J].广西大学.20~25, 2020

Shen Yang; Liu Wei; Zhu Zhe; Liu Shuangmei; Cao Yanyan; Yan Lei; Chen Liang.Application of a preoperative pain management mode based on instant messaging software in elderly hip fracture patients: a randomized controlled trial[J].PubMed期刊, 27~28,2023

致谢

在完成基于 Uni - app 的即时通讯项目毕业论文之际，满心感恩，诸多情谊，难以言表。

我要诚挚感谢我的导师，从项目选题的迷茫阶段，到系统结构设计的反复雕琢，再到测试环节的细致把控，导师始终以渊博的专业知识和丰富的经验为我指引方向。每当我在研究中陷入困境，导师总能耐心倾听，给出建设性的意见，助我拨云见日。导师严谨的治学态度与精益求精的精神，如明灯照亮我学术探索之路，激励我不断追求卓越，为论文的顺利完成奠定了坚实基础。

还要感谢我的同窗好友们，在项目开发过程中，我们相互交流、彼此启发。面对技术难题时，大家围坐一起热烈讨论，思维的火花在碰撞中绽放。那些一起熬夜调试代码、攻克难关的日子，成为我学术生涯中难忘的回忆。他们的陪伴与鼓励，让枯燥的研究工作充满温暖与力量，也让我在团队协作中收获了宝贵的经验。

我的家人，是我永远的坚强后盾。在漫长的研究时光里，他们默默承担生活琐事，给予我无条件的支持与关爱，让我能够心无旁骛地投入到学业中。他们的牵挂与期待，是我前进的不竭动力，每当我感到疲惫想要放弃时，一想到家人，便又充满斗志。

同时，我也要感谢为 Uni - app 开发框架以及即时通讯技术发展做出贡献的开源社区和开发者们。正是站在他们的智慧结晶之上，我才能顺利开展项目研究，借鉴优秀的设计理念与成熟的技术方案，少走了许多弯路。

在此，向所有给予我帮助的人致以最衷心的感谢。这份感恩将化作前行的动力，激励我在未来的学术与人生道路上，不断进取，勇攀高峰。

说明

- 1、疑似AIGC概率越大，风险等级越高，代表AI生成可能性越大。
- 2、系统检测结果仅作为校方评价学生论文的参考依据之一，建议不作为最终评价依据。学生论文最终评价，以校方评价标准和评价结果为准。



关注微信公众号

官方网站: www.gocheck.cn

客服热线: 400-699-3389

客服QQ: 800113999